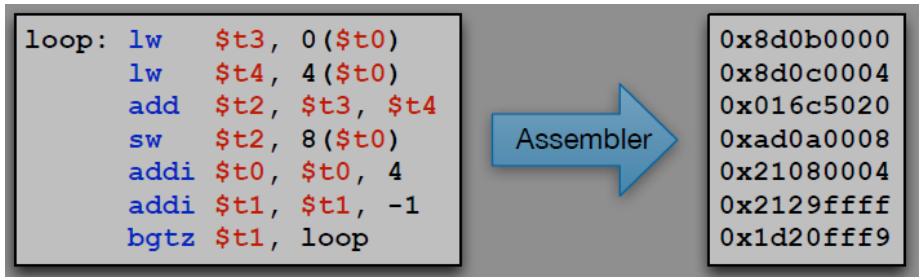
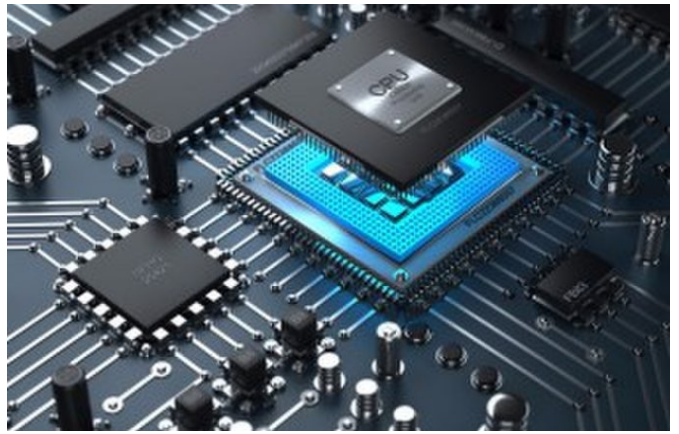




Lecture 08:
CPU Memory Access and Addressing

CMPSC 64, SPRING 2026

ZIAD MATNI, PH.D.

UNIVERSITY OF CALIFORNIA, SANTA BARBARA

Administrative

Current assignment #4 is due on Monday

- Programming assignment – best not to leave until the last minute

New readings for this week!

- **Handout 3 on Memory Access/Addressing** **(read/view by Friday Lab)**
- **Handout 4 on Instruction Assembly** **(read/view by Monday)**
- ~~**Video on Instruction Assembly**~~ **leave that for next week…**

~~*Worksheet for this lecture available on Canvas*~~ *(sorry)*

Reminder: Office Hours 4 U!!!!!

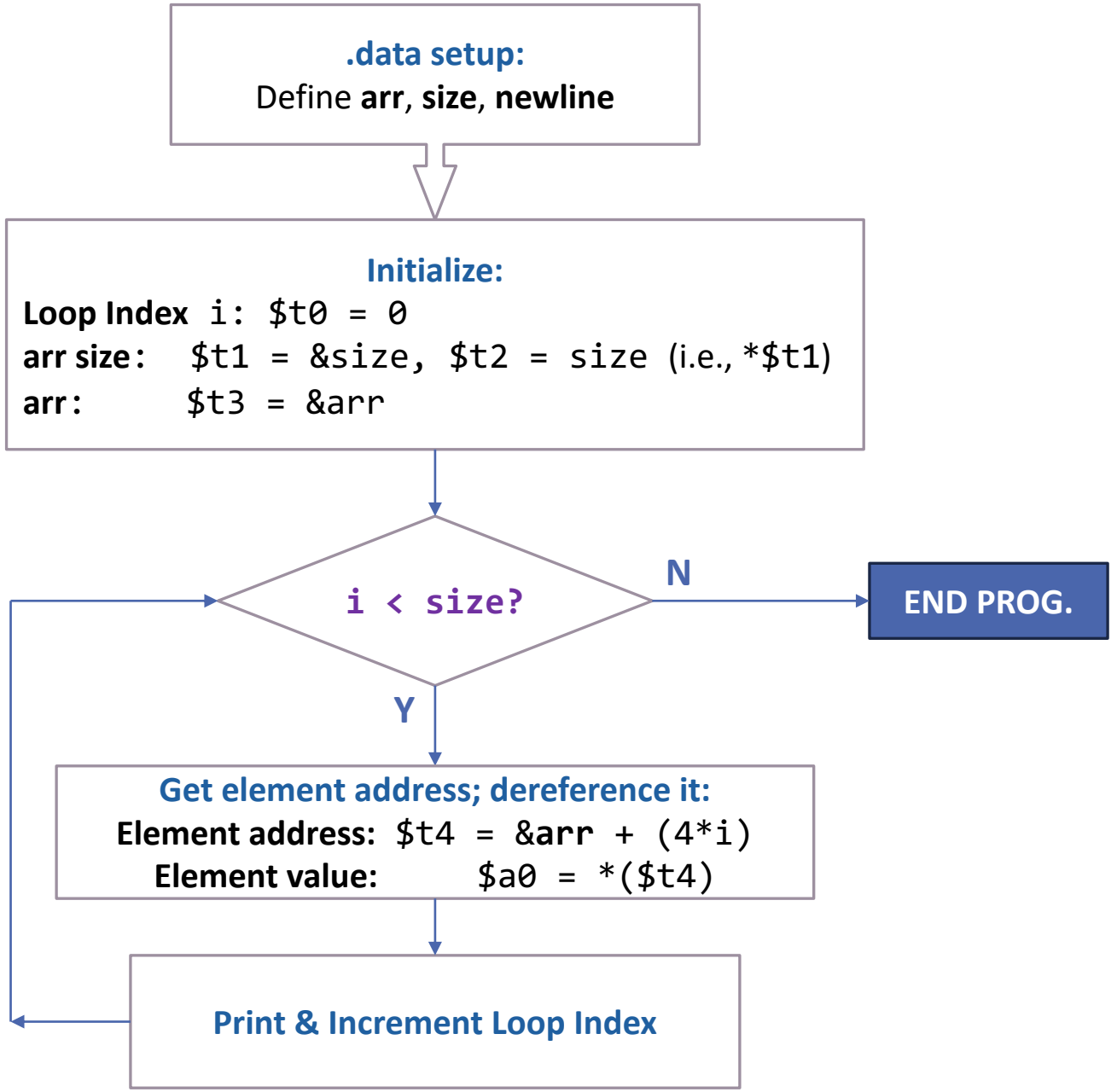
Arrays of Integers
EXAMPLE: print_array1.asm

```
int arr[] = {5, 32, 87, 95, 286, 386};
static int size = 6;

for (int i = 0; i < size; i++) {
      cout << arr[i] << endl;
}
```

```
int arr[] = {5, 32, 87, 95, 286, 386};
static int size = 6;

for (int i = 0; i < size; i++) {
      cout << arr[i] << endl;
}
```



print_array1.asm

```
#   int arr[] = {5, 32, 87, 95, 286, 386};
#   static int size = 6;
#   for (int i = 0; i < size; i++) {
#       cout << arr[i] << endl; }
.data
newline:  .asciiz "\n"
arr:      .word 5 32 87 95 286 386
size:     .word 6
.text
main:     # let $t0 be index i; initialize i to 0
          li $t0, 0
          # get size, put result in $t2
          la $t1, size        # $t1 = &size
          lw $t2, 0($t1)      # $t2 = *(&size)
          # get the base address of arr
          la $t3, arr         # $t3 = &arr
loop:
          # if i < size, set $t9=1
          slt $t9, $t0, $t2
          # jump out if not true (i.e. $t9=0)
          beq $t9, $zero, exit
          # figure out where in the array we need
          # to read from. This is going to be the array
          # address + (index << 2). The shift is
          # multiplication by four to index bytes
          # as opposed to words.
          sll $t4, $t0, 2
          addu $t4, $t4, $t3

          # print out arr[i], with a newline
          lw $a0, 0($t4)
          li $v0, 1
          syscall

          la $a0, newline
          li $v0, 4
          syscall

          # increment index
          addi $t0, $t0, 1

          # restart loop
          j loop

exit:
          # exit the program
          li $v0, 10
          syscall
```

Arrays of Integers using Pointers
EXAMPLE: print_array2.asm

```
int arr[]   = {5, 32, 87, 95, 286, 386};
int size    = 6;
int* p;

for (p = arr; p < arr + size; p++) {
      cout << *p << endl;
}
```

Notes on print_array2.asm

```
int arr[]   = {5, 32, 87, 95, 286, 386};
int size    = 6;
int* p;

for (p = arr; p < arr + size; p++) {
      cout << *p << endl;
}
```

Same result, but *more efficient* than the previous example – why is that??

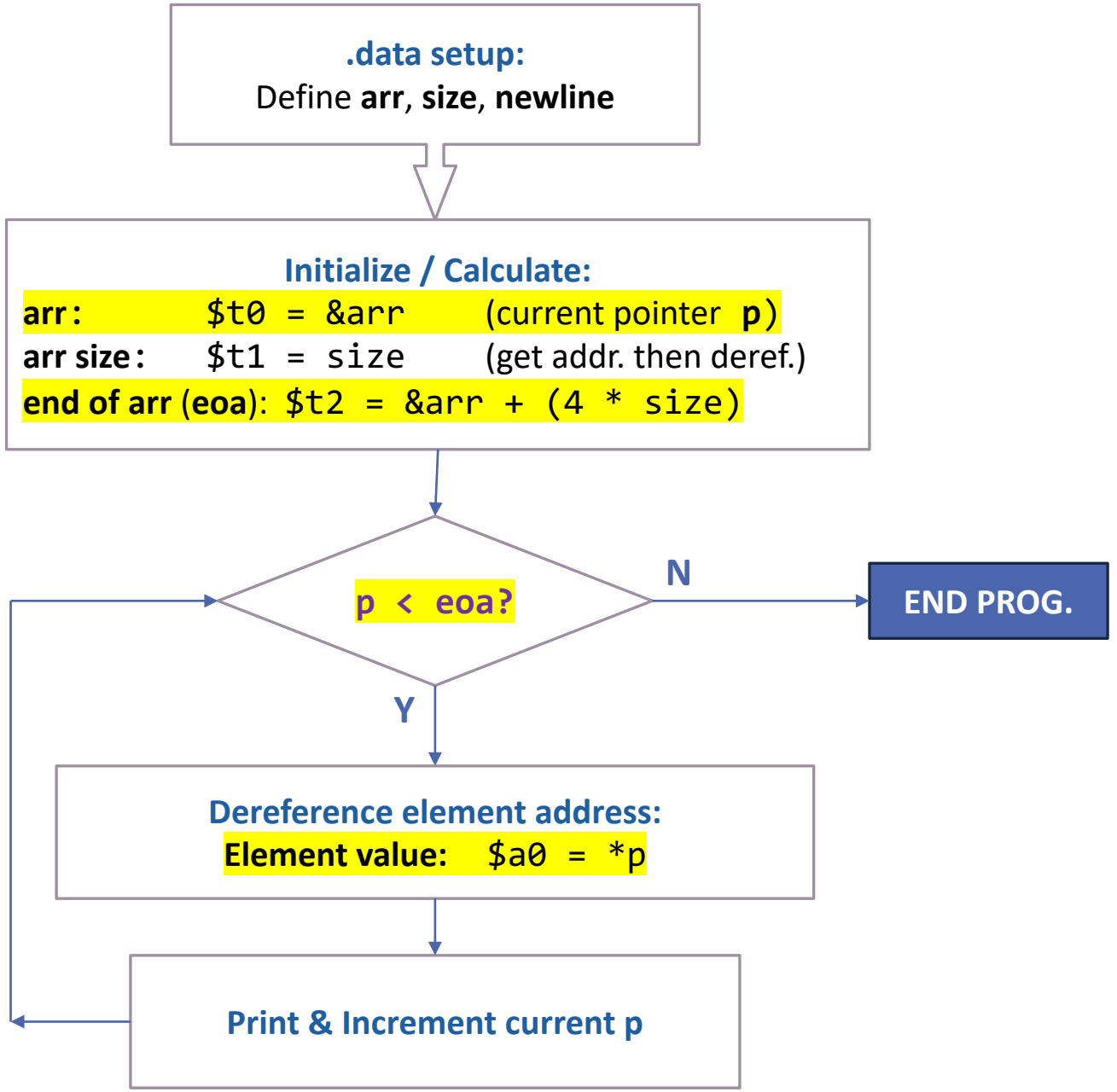
ANSWER: Because the loop works with actual *memory addresses not indices*

- i.e., not plain indices --- recall Example 1's for loop used index **i**
- We can then *dereference* the pointer only when we want to print it out

This is also the sort of thing an optimized compiler can do with a HLL program

```
int arr[]   = {5, 32, 87, 95, 286, 386};
int size    = 6;
int* p;

for (p = arr; p < arr + size; p++) {
      cout << *p << endl;
}
```



print_array2.asm

```
#   int arr[] = {5, 32, 87, 95, 286, 386};
#   static int size = 6;
#   for (int* p = arr; p < arr + size; p++) {
#       cout << *p << endl; }
.data
newline:  .asciiz "\n"
arr:      .word 5 32 87 95 286 386
size:     .word 6
.text
main:     la $t0, arr         # This is var. p
          la $t1, size
          lw $t1, 0($t1)      # t1 = 6: no. of elements
          sll $t1, $t1, 2     # t1 = 24: no. of bytes
# Make t2 = arr address + size in bytes:
# This is the address of the END of the array (in bytes):
          addu $t2, $t0, $t1
loop:
          # see if current p < END of array address
          slt $t1, $t0, $t2      # is it ok to reuse $t1???
          # jump to exit if not true
          beq $t1, $zero, exit
```

```
otherwise:
     # dereference p and print it out w/ a newline:
     lw $a0, 0($t0)      # a0 = value at current p
     li $v0, 1
     syscall

     la $a0, newline
     li $v0, 4
     syscall

     # increment pointer p by 4
     # b/c each element in array is 4 bytes long
     addiu $t0, $t0, 4

     # restart loop
     j loop

exit:
     # exit the program
     li $v0, 10
     syscall
```

Explain how this is more efficient than print_array1.asm?

Worksheet

DEMO!!!

To Dos

Work on current assignment

Do the readings

Go to lab on Friday!

</LECTURE>